Guía de Ejercicios N°5

EIE 434 – Programación 2 | Escuela de Ingeniería Eléctrica

Unidad: 1 **Semestre:** 1er Semestre 2026 **Docente:** Guillermo Cid Ampuero

Ayudante: Amaro Díaz

1. Objetivos

Esta guía tiene como objetivo repasar y reforzar los contenidos de la clase teórica sobre el **Manejo de Errores y Excepciones** mediante la resolución de problemas aplicados a la lectura de sensores, control de motores y análisis de datos en Python.

2. Ejercicios

Ejercicio 1 – Estación Meteorológica – Validación de Datos Seriales

Se está recibiendo una lista de lecturas de temperatura desde un módulo WiFi. Ocasionalmente, el módulo envía datos corruptos o cadenas de texto vacías debido a interferencias.

a) Cree una función llamada `limpiar_datos(lecturas)` que reciba una lista de datos mixtos (números y strings).

b) Itere sobre la lista e intente convertir cada valor a flotante (`float`) utilizando un bloque `try-except`.

c) Si ocurre un `ValueError` o `TypeError`, capture la excepción, imprima un mensaje indicando el dato defectuoso, y omítalo de la lista resultante.

d) Pruebe su función con la siguiente lista de prueba: `[22.5, '23.1', 'Error', None, 21.8, ' ']`.

e) Calcule y retorne el promedio de las temperaturas válidas obtenidas.

Ejercicio 2 – Control de Motores – Operaciones Críticas y Aserciones

Para calcular el rendimiento de un motor de corriente continua, se debe dividir la potencia de salida mecánica por la potencia de entrada eléctrica.

a) Escriba una función `calcular_eficiencia(p_salida, p_entrada)`.

b) Utilice la instrucción `assert` al inicio de la función para garantizar que ambas potencias sean mayores o iguales a cero. Si no lo son, debe arrojar el mensaje "Las potencias no pueden ser negativas".

c) Dentro de un bloque `try-except`, calcule la eficiencia como el cociente entre `p_salida` y `p_entrada` multiplicado por 100.

d) Capture la excepción `ZeroDivisionError` en caso de que la potencia de entrada sea

cero, retornando el mensaje .Error: Motor apagado o sin consumo".

e) Pruebe su función con los pares de valores: $(150, 200)$, $(100, 0)$ y $(-50, 100)$.

Ejercicio 3 – Calibración de Sensores – Accesos Seguros y Gráficos

Se dispone de un diccionario que almacena los factores de calibración históricos para distintos sensores del laboratorio: `calibracion = {'temp': [1.02, 0.98, 1.04, 1.01], 'hum': [1.05, 0.99, 1.02], 'presion': [1.0, 0.99, 1.01, 0.98]}`.

a) Solicite al usuario (mediante `input`) que ingrese el nombre del sensor que desea consultar.

b) Solicite también el índice del factor de calibración que necesita revisar. Recuerde convertir esta entrada a entero.

c) Utilice un único bloque `try` con múltiples `except` para intentar acceder y mostrar en pantalla el valor solicitado: `calibracion[sensor][indice]`.

d) Capture la excepción `KeyError` si el usuario ingresa un sensor que no está en el diccionario.

e) Capture la excepción `IndexError` si el usuario ingresa un índice mayor a la cantidad de valores disponibles para ese sensor.

f) Capture `ValueError` por si el usuario ingresa letras en lugar de un número para el índice.

g) Si no ocurren errores, utilice `matplotlib.pyplot` para generar un gráfico de barras que muestre todos los factores de calibración del sensor seleccionado.

h) En el gráfico, destaque con un color diferente (ej. naranjo) la barra correspondiente al índice consultado por el usuario. Agregue título, etiquetas a los ejes y una grilla.

Ejercicio 4 – Registro de Datos – Archivos y Bloque Finally

Durante la captura de datos en terreno, es crucial asegurar que los archivos de registro se cierren correctamente, incluso si el programa falla a la mitad de la lectura.

a) Escriba un script que intente abrir un archivo llamado `parametros_robot.txt` en modo lectura (`'r'`).

b) En el bloque `try`, simule un error forzado (por ejemplo, intente realizar la operación `10 / 0`) justo después de abrir el archivo.

c) Capture la excepción `FileNotFoundError` en caso de que el archivo no exista en su directorio, imprimiendo 'Archivo de configuración no encontrado'.

d) Capture la excepción `ZeroDivisionError` generada en el paso (b).

e) Utilice un bloque `finally` para asegurarse de cerrar el archivo usando el método `.close()` solo si el archivo fue abierto exitosamente. *Pista: puede declarar la variable del archivo como `None` antes del try.*

f) Opcional: Reescriba este mismo ejercicio utilizando el gestor de contexto `with` y observe cómo se reduce la necesidad del bloque `finally`.

Nota: Solución de la Clase: Repositorio del Curso